Node JS-1

Introduction

What is Node.js?

- Node.js is an open source server environment. It allows to run Java Script on the server. (It is not a framework or programming language)
- Node.js is free
- **Runtime environment** for building highly scalable server-side applications using JS.
- Node.js often use for building back-end services like APIs, Web app, Mobile app.
- Node.js runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)

Why Node.js?

Node.js uses asynchronous programming!

A common task for a web server can be to open a file on the server and return the content to the client.

Here is how Node.js handles a file request:

1. Sends the task to the computer's file system.
2. Ready to handle the next request.
3. When the file system has opened and read the file, the server returns the content to the client.

Node.js eliminates the waiting, and simply continues with the next request.

Node.js runs single-threaded, non-blocking, asynchronous programming, which is very memory efficient.

What Can Node.js Do?

- Node.js can generate dynamic page content.
- Node.js can create, open, read, write, delete, and close files on the server.
- Node.js can collect form data.
- Node.js can add, delete, modify data in your database.

What is a Node.js File?

- Node.js files contain tasks that will be executed on certain events.
- A typical event is someone trying to access a port on the server.
- Node.js files must be initiated on the server before having any effect.
- Node.js files have extension ".js".

Setup

Steps to Install Node.js

1. Visit the official website
   https://nodejs.org
2. Download the **LTS version(Long Term Support)** of Node.js.
3. Run and install the setup file.
4. Verify installation using the terminal:

Check Node.js version:

```
node -v
```

Check npm version:

```
npm -v
```

- **Latest LTS version of Node.js is 24.x (LTS)** – recommended for production applications.
- **Latest Current version is 25.x** – includes the newest features but may change frequently.
- The Node.js installer includes the NPM(Node Package Manager).
- **NPM is the package manager** for the Node JS platform. It puts modules in place so that node can find them, and manages dependency conflicts intelligently.

Node.js Architecture

Node.js follows a **single-threaded, event-driven architecture** that allows it to handle multiple requests efficiently without creating multiple threads.

Key Components

1. **Client Requests**
   Requests sent by users through a browser or application.
2. **Event Queue**
   Stores incoming requests in a queue.
3. **Event Loop**
   Continuously checks the queue and processes requests.
4. **Worker Threads**
   Handle heavy operations such as file access, database queries, or network operations.

Asynchronous Programming in Node.js

Node.js supports **asynchronous programming**, meaning tasks can run without blocking the main execution thread.

Blocking vs Non-Blocking Operations in Node.js

In Node.js, operations are classified as **blocking** or **non-blocking** depending on whether the system waits for a task to complete before moving to the next task.

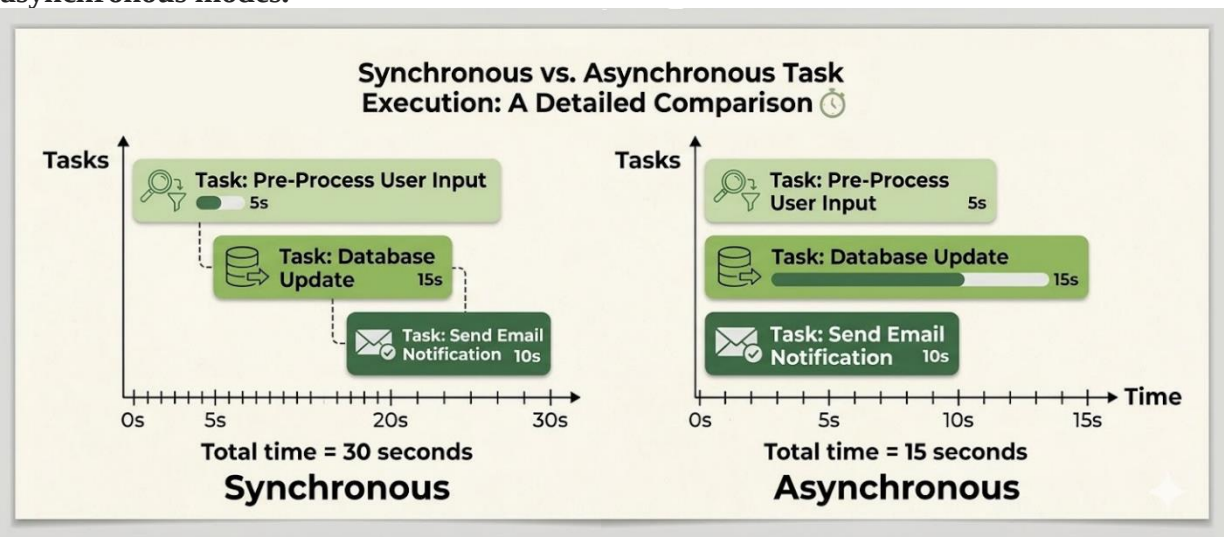
1. Blocking Operations

- A **blocking operation** stops the execution of the program until the current task is completed.
- When a request arrives, the system waits until the task finishes before processing another request.
- Reading a file from disk in a blocking way. If a user requests a large file, the server waits until the file is completely read before handling any other request.

2. Non-Blocking Operations

- A **non-blocking operation** allows the program to continue executing other tasks while waiting for the current task to complete.
- The system starts a task and immediately moves to the next request without waiting for completion.
- Reading a file asynchronously. User A requests a file. The server starts reading the file and immediately processes User B's request. When the file reading is completed, the response is sent to User A.

The illustration below compares the time taken for tasks in both synchronous and asynchronous modes.



Callback using simple JS functions like setInterval(), setTimeout()

Why do we need Callback Functions?
JavaScript runs code sequentially in top-down order. However, there are some cases that code runs (or must run) after something else happens and also not sequentially. This is called **asynchronous programming**.

Callbacks make sure that a function is not going to run before a task is completed but will run right after the task has completed. It helps us develop asynchronous JavaScript code and keeps us safe from problems and errors.

In JavaScript, the way to create a callback function is to pass it as a parameter to another function, and then to call it back right after something has happened or some task is completed.

Syntax: function function_name(argument, callback)

How to create a Callback? - use of setTimeout()

JavaScript setTimeout() Method:

This method executes a function, after waiting a specified number of milliseconds.

To understand what explained above, lets start with a simple example. We want to log a message to the console but it should be there after 3 seconds.

```
const message = function() {
   console.log("This message is shown after 3 seconds");
}
setTimeout(message, 3000);
```

There is a built-in method in JavaScript called “setTimeout”, which calls a function or evaluates an expression after a given period of time (in milliseconds). So here, the “message” function is being called after 3 seconds have passed. (1 second = 1000 milliseconds)
In other words, the message function is being called after something happened (after 3 seconds passed for this example), but not before. So the message function is an example of a callback function.

What is an Anonymous Function?
Alternatively, we can define a function directly inside another function, instead of calling it. It will look like this:

```
setTimeout(function() {
   console.log("This message is shown after 3 seconds");
}, 3000);
```

As we can see, the callback function here has no name and a function definition without a name in JavaScript is called as an “anonymous function”. This does exactly the same task as the example above.

Callback as an Arrow Function

If you prefer, you can also write the same callback function as an ES6 arrow function, which is a newer type of function in JavaScript:

Write js code that display hello in capital after 5 second

```
setTimeout(() => {
console.log("hello".toUpperCase())
}, 5000);
```

JavaScript setInterval() Method:

The setInterval() method repeats a given function at every given time interval. For example clock

Display clock using setInterval Method

```
function updateTime() {
  // Get the current time in HH:MM:SS format for India timezone
  const timeString = new Date().toLocaleTimeString();
  console.log(timeString);
}
updateTime();

// Call updateTime every second (1000 milliseconds)
setInterval(updateTime, 1000);
```

Some callback examples

1. Display content on browser after 5 seconds

```
<html>
  <head>
  </head>
  <body>
    <p id="id"></p>
    <script>
      setTimeout(myfun,5000);
      function myfun()
      {
        document.getElementById("id").innerHTML="LJU";
      }
    </script>
  </body>
</html>
```

2. Display addition of two number on browser using callback function

```
<html>
<head>
</head>
<body>
  <p id="demo"></p>
  <script>
    function mydisplay(sum)
    {
      document.getElementById("demo").innerHTML="<b>"+ sum +"</b>";
    }
    function mycals(num1,num2,mycallback)
    {
      sum=num1+num2;
      mycallback(sum);
    }
    mycals(13,15,mydisplay);
  </script>
</body></html>
```

Output : 28

3. Initialize two variables and increment both the variables each time and display the addition of both the variables at interval of 1 second.

```
<html>
<body>

<p id="p1"></p>

<script>
let a = 2, b = 5;

setInterval(() => {
document.getElementById("p1").innerHTML = ++a + ++b;
}, 1000);

</script>

</body>
</html>
```

Output : Display 9 and then incremented

4. Write a js code that display ”Hello” with increasing font size in interval of 50ms in blue colour and it should stop when font size reaches to 50px.

```
<html>
   <body>
      <p id="demo" style="color:blue"></p>
      <script>
         size = 15;
         function add() {

            obj = document.getElementById("demo");
            obj.innerHTML = "hello";
            obj.style.color ="blue";
            obj.style.fontSize = size + “px”;
            if (size <= 50) {
               size++;
            }
         }
         setInterval(add, 50);
      </script>
   </body>
</html>
```

5. Write code to increase the font size at interval of 50 ms and it should stop increasing when the font size reaches to 50px. This task should be performed when you click on "Increase button" on browser. (Default font size 15px)

```
<html>
   <head>
      <style>
         p{
            color:blue;
         }
      </style>
   </head>
   <body>
      <p id="p1"> Hello</p>
      <button onclick="fun2()">font-size</button>
      <script>
      font="15";
      function fun(font)
      {
        document.getElementById("p1").style.fontSize=font;
      }
      function fun2()
      {
         setInterval(
            function()
            {
               if(font<=50)
               {
                  fun(font++);
               }
            },50 );
      }
   </script> </body></html>
```

6 Write code to perform the tasks as asked below.

- **Add three buttons.**
- **Increase button to increase the fonts. It should stop increasing the fonts when the font size reaches to 200px or stop button is clicked.**
- **Decrease button to decrease the fonts. It should stop decreasing the fonts when the font size reaches to 20px or stop button is clicked.**
- **Stop button to stop increasing or decreasing the fonts.**
- **Increasing/decreasing interval is of 100 ms.**

• **(Default font size = 50px)**

```
<html>
<body>
<p id="p1" style="font-size: 50px;"> Hello</p>
<button onclick="inc()">Increase</button>
<button onclick="dec()">Decrease</button>
<button onclick="stop()">stop</button>
<script>
font=50;
function fun(font){
document.getElementById('p1').style.fontSize=font+'px';
}
function inc()
{
test = setInterval(()=>{
if(font<100){ fun(++font);
}
},100);
}
function dec()
{
test =setInterval(()=>{
if(font>15){ fun(--font);}
},100);
}
function stop() { clearInterval(test); }
</script>
</body>
</html>
```

Nodejs Core Modules

Node.js is a lightweight framework. The core modules include bare minimum functionalities of Node.js. These core modules are compiled into its binary distribution and loaded automatically when the Node.js process starts. However, you need to import the core module first in order to use it in your application.

To include a module, use the require() function with the name of the module:

```
var module = require('module_name');
```

The require() function will return an object, function, property or any other JS type dependency on what the specified module returns.

❖ File System Module

The Node.js file system module allows you to work with the file system on your computer. To include the File System module, use the require() method:

```
var fs = require('fs');
```

Operation	Async Method	Sync Method	Description
Read file	fs.readFile()	fs.readFileSync()	Reads the contents of a file
Write file	fs.writeFile()	fs.writeFileSync()	Creates or overwrites a file
Append file	fs.appendFile()	fs.appendFileSync()	Adds data to a file/ If not then create a new file
Copy file	fs.copyFile()	fs.copyFileSync()	Copies a file
Rename	fs.rename()	fs.renameSync()	Renames a file or directory
Delete	fs.unlink()	fs.unlinkSync()	Deletes a file
Create directory	fs.mkdir()	fs.mkdirSync()	Creates a directory
Remove directory	fs.rmdir()	fs.rmdirSync()	Removes a directory

Synchronous Approach -Blocking Mode

1. **mkdirSync()**: The fs.mkdirSync() method is used to create a directory Synchronously.

```
Syntax:
fs.mkdirSync( path, options )
```

Parameters: This method accept two parameters as mentioned above and described below:
- **path:** The path at which directory is to be created. It can be string, buffer, etc.
- **options**: It is an optional parameter which determines how to create directory like recursively, etc.

2. **writeFileSync().** The fs.writeFileSync() creates a new file if the specified file does not exist.

```
Syntax:
fs.writeFileSync( file, data, options )
```

Parameters:
- **file:** It is a string, Buffer, URL or file description integer that denotes the path of the file where it has to be written.
- **data:** It is a string, Buffer, TypedArray or DataView that will be written to the file.

3. **readFileSync() :**The fs.readFileSync() method is an inbuilt application programming interface of the fs module which is used to read the file and return its content.
In the fs.readFileSync() method, we can synchronously read files, i.e. we are telling node.js to block other parallel processes and do the current file reading process.

```
Syntax:
fs.readFileSync( path, options )
```

Parameters:
- **path:** It takes the relative path of the text file. The path can be of URL type. The file can also be a file descriptor. If both the files are in the same folder just give the filename in quotes.
- **options:** It is an optional parameter that contains the encoding and flag, the encoding contains data specification.
  Its default value is null which returns the raw buffer and the flag contains an indication of operations in the file. Its default value is ‘r’.

Note: To read data always use
Mention “utf-8” as option. UTF-8 is the World Wide Web’s most common character encoding.
In Node.js, the Buffer.toString() method is used to decode or convert a buffer to a string, according to the specified character encoding type.
Converting a buffer to a string is known as encoding, and converting a string to a buffer is known as decoding.

fs.readFileSync("node/write.txt",”utf-8”);
or
data= fs.readFileSync("node/write.txt");
console.log(data.toString());

If no encoding is specified, then the raw buffer is returned.

4. **appendFileSync() :**The fs.appendFileSync() method is used to synchronously append the given data to a file. A new file is created if it does not exist. The optional options parameter can be used to modify the behavior of the operation.

Syntax:
fs.appendFileSync(path, data, [options])

5. **copyFileSync()** The fs.copyFileSync() method is used to synchronously copy a file from a source path to a destination path. It will overwrite the destination file if it already exists (unless flags are used to prevent it).

Syntax:
fs.copyFileSync(src, dest[, mode])

6. **renameSync()** :The fs.renameSync() method is used to synchronously rename a file at the given old path to the given new path. It will overwrite the destination file if it already exists.

Syntax:
fs.renameSync(oldPath, newPath)

7. **unlinkSync():** The fs.unlinkSync() method is used to synchronously remove a file or symbolic link from the filesystem. This function does not work on directories, therefore it is recommended to use fs.rmdir() to remove a directory.

Syntax:
fs.unlinkSync(path)

8. **rmdirSync():** The fs.rmdirSync() method is used to synchronously remove a directory. It only works on empty directories (deprecated for recursive use ,**So use rmSync(path)**).

Syntax:
fs.rmdirSync(path)

Example: Perform Write and Read operation in Sync.

```
 var ps=require("fs");
// To Write File in Sync
ps.writeFileSync("Hello.txt","Hello World")
// To Read File in Sync
var data=ps.readFileSync("Hello.txt");
//console.log(data) – Provides Buffer data
//Use .toString() Method to convert this Buffer object to a string
console.log(data.toString());
console.log("Program ended");
```

Output:

```
Hello World
Program ended
```

Asynchronous Approach – Non-blocking Mode

In morden programming use of asynchronous file system functions provided by the **fs** module to perform operations like reading, writing, and deleting files. These functions have the same names as their synchronous counterparts, but with 'Sync' appended to the function names.

For example fs.writeFile(file, data, **callback**) will become **"**fs.writeFile**Sync**(file, data)"

1. Create files fs.writeFile()

Syntax:

```
fs.writeFile( file, data, options, callback )
```

Parameters:
This method accepts four parameters as mentioned above and described below:

- **file:** It is a string, Buffer, URL or file description integer that denotes the path of the file where it has to be written. Using a file descriptor will make it behave similar to fs.write() method.
- **data:** It is a string, Buffer, TypedArray or DataView that will be written to the file.
- **options(optional):** It is an string or object that can be used to specify optional parameters that will affect the output. It has three optional parameter:

- **encoding:** It is a string value that specifies the encoding of the file. The default value is 'utf8'.
- **mode:** It is an integer value that specifies the file mode. The default value is 0o666.
- **flag:** It is a string value that specifies the flag used while writing to the file. The default value is 'w'.
- **callback:** It is the function that would be called when the method is executed.
- **err:** It is an error that would be thrown if the operation fails.

Example

```
const fs = require('fs');

fs.writeFile('data.txt', 'Hello Node.js', (err) => {
   if (err) throw err;
   console.log("File written successfully");
});
```

2. Read files fs.readFile()

Syntax:

```
fs.readFile( filename, encoding, callback_function )
```

Parameters: The method accepts three parameters as mentioned above and described below:

- **filename:** It holds the name of the file to read or the entire path if stored at another location.
- **encoding:** It holds the encoding of the file. Its default value is **'utf8'**.
- **callback_function:** It is a callback function that is called after reading of file. It takes two parameters:
    - **err:** If any error occurred.
    - **data:** Contents of the file.

Return Value: It returns the contents/data stored in file or error if any.

Example :

```
const fs = require('fs');
fs.readFile('data.txt', 'utf-8', (err, data) => {
   if (err) throw err;
   console.log(data);
});
```

3. Update files fs.appendFile()

Syntax:

```
fs.appendFile( path, data, options, callback )
```

Parameters:

This method accepts four parameters as mentioned above and described below:

- **path:** It is a String, Buffer, URL or number that denotes the source filename or file descriptor that will be appended to.
- **data:** It is a String or Buffer that denotes the data that has to be appended.
- **Options (optional):** It is a string or an object that can be used to specify optional parameters that will affect the output. It has three optional parameters:
    - **encoding:** It is a string which specifies the encoding of the file. The default value is 'utf8'.
    - **mode:** It is an integer which specifies the file mode. The default value is '0o666'.
    - **flag:** It is a string which specifies the flag used while appending to the file. The default value is 'a'.
- **callback:** It is a function that would be called when the method is executed.
    - **err:** It is an error that would be thrown if the method fails.

Example

```
const fs = require('fs');
// Async
fs.appendFile('data.txt', '\nNew line added', (err) => {
   if (err) throw err;
   console.log("Content appended");
});
```

4. Delete files fs.unlink()

Syntax:

```
fs.unlink( path, callback )
```

Parameters: This method accepts two parameters as mentioned above and described below:

- **path:** It is a string, Buffer or URL that**,** represents the file or symbolic link that has to be removed.
- **callback:** It is a function that would be called when the method is executed.
- **err:** It is an error that would be thrown if the method fails.

Example

```
const fs = require('fs');
// Async
fs.unlink('data.txt', (err) => {
   if (err) throw err;
   console.log("File deleted");
});
```

5. Rename files fs.rename()

Syntax:

```
fs.rename(oldPath, newPath, callback)
```

Parameters: This method accepts three parameters as mentioned above and described below:
• **oldPath:** It is a string, Buffer or URL that represents the current file path.
• **newPath:** It is a string, Buffer or URL that represents the new file path.
• **callback:** It is a function that would be called when the method is executed.
• **err:** It is an error that would be thrown if the method fails.

Example

```
const fs = require('fs');
// Async
fs.rename('old.txt', 'new.txt', (err) => {
    if (err) throw err;
    console.log("File renamed");
});
```

6. Copy files fs.copyFile()

Syntax:

```
fs.copyFile(src, dest, callback)
```

Parameters: This method accepts three parameters as mentioned above and described below:
• **src:** It is a string, Buffer or URL that represents the source file path.
• **dest:** It is a string, Buffer or URL that represents the destination file path.
• **callback:** It is a function that would be called when the method is executed.
• **err:** It is an error that would be thrown if the method fails.

Example

```
const fs = require('fs');
// Async
fs.copyFile('source.txt', 'destination.txt', (err) => {
    if (err) throw err;
    console.log("File copied");
});
```

7. Create directory fs.mkdir()

Syntax:

```
fs.mkdir(path, options, callback)
```

Parameters: This method accepts three parameters as mentioned above and described below:
• **path:** It is a string, Buffer or URL that represents the directory path to be created.
• **options:** It is an optional parameter that can specify recursive creation and mode.
• **callback:** It is a function that would be called when the method is executed.
• **err:** It is an error that would be thrown if the method fails.

Example

```
const fs = require('fs');
// Async
fs.mkdir('myFolder', (err) => {
   if (err) throw err;
   console.log("Folder created");
});
```

8. Remove directory fs.rmdir()

Syntax:

```
fs.rmdir(path, options, callback)
```

Parameters: This method accepts three parameters as mentioned above and described below:
• **path:** It is a string, Buffer or URL that represents the directory path to be removed.
• **options:** It is an optional parameter (used for recursive deletion in older versions).
• **callback:** It is a function that would be called when the method is executed.
• **err:** It is an error that would be thrown if the method fails.

Example

```
const fs = require('fs');
// Async
fs.rmdir('myFolder', (err) => {
   if (err) throw err;
   console.log("Folder removed");
});
```

File Module (CRUD)

Write node Example with File system methods in Synchronous Mode.

1. **To create folder**
2. **Create one file inside that folder**
3. **Append some data to that file.**
4. **Read data from the file**
5. **Rename that file**
6. **Delete File**
7. **Delete Folder**

```
var fs=require("fs");

fs.mkdirSync("Hello");                          //Generate Folder named Hello

fs.writeFileSync("Hello/user.txt","Hello");     // Insert File user.txt in Hello folder with data

fs.appendFileSync("Hello/user.txt","\nWorld"); // append the data

var data=fs.readFileSync("Hello/user.txt","utf-8"); // Read data

fs.renameSync("Hello/user.txt","Hello/user1.txt"); // Rename file name

console.log(data.toString());

fs.unlinkSync("Hello/user1.txt");               // Delete File

fs.rmdirSync("Hello");                          // Delete Folder (Folder Must be empty)
```

Output:
Hello
World

What is UTF-8 encoding?

UTF-8 is a type of character encoding that is commonly used to represent text in computer systems. It stands for "**Unicode Transformation Format - 8 bits**" and is a variable-length encoding that can represent every character in the Unicode character set.

In Node.js, UTF-8 is the default encoding for reading and writing text files. When you read a text file using a method like fs.readFileSync() or fs.readFile(), you can specify the encoding as UTF-8 to ensure that the text is correctly interpreted.

It is widely used on the web and in many computer systems because it is efficient for handling ASCII characters (which use only one byte), but can also represent characters from non-Latin scripts (which require more than one byte

Write a Nodejs script to take 5 single digit elements separated by white space in .txt file using .sort method. Print sorted array of these 5 elements on Node Js server.

//string format

```
var ps=require("fs");
ps.writeFileSync("s1.txt","5 9 6 1 2 0");
data=ps.readFileSync("S1.txt","utf-8");
data=data.split(" ");
data.sort();
console.log(data);

Output:
['0', '1', '2', '5', '6', '9']
```

//integer format

```
var ps=require("fs");
ps.writeFileSync("task.txt","5 9 6 1 2 0");
data=ps.readFileSync("task.txt","utf-8")
  console.log(data);
data=data.split(" ");
  console.log(data);
for(i=0;i<data.length;i++){
    data[i]=parseInt(data[i]);}
d1=data.sort();
  console.log(d1);
```

Output:

```
5 9 6 1 2 0
[ '5', '9', '6', '1', '2', '0' ]
[ 0, 1, 2, 5, 6, 9 ]
```

Note: For More than 2 digits along with sign weight.

```
const array = [1, 12, 2];
array.sort((a, b) => a - b);
console.log(array); // Output: [1, 2, 12]
```

In this example, the comparison function (a, b) => a - b subtracts b from a. If the result is negative, a comes before b in the sorted order. If it's positive, b comes before a. If it's zero, their order remains unchanged.

By providing this comparison function, the sort() method sorts the numbers in ascending order based on their numerical values, resulting in [1, 2, 12]. Else due to lexicographical order your answer will not correct.

Write a node.js script to write and copy contents of one file to another file. Data should be fetched from Source.txt and insert to destination.txt in Synchronous mode.

```
var ps=require("fs");

ps.writeFileSync("source.txt","ABC");

ps.appendFileSync("source.txt","DEF");

data=ps.readFileSync("Source.txt","utf-8");

ps.writeFileSync("destination.txt",data);

data1=ps.readFileSync("destination.txt","utf-8");

console.log(data.toString());
```

Output:

ABCDEF

Write a node.js script to copy contents of one file to another file. Data should be fetched from Source.txt and insert to destination.txt. And read data from destination file in the end. Perform all these tasks asynchronously.

```
const fs = require('fs');

// Step 1: Write data to Source.txt
fs.writeFile('Source.txt', 'ABC', (err) => {
    if (err) throw err;
    console.log("Source file created.");

    // Step 2: Read data from Source.txt
    fs.readFile('Source.txt', 'utf-8', (err, data) => {
        if (err) throw err;
        console.log("Data read from source:", data);

        // Step 3: Write data to destination.txt
        fs.writeFile('destination.txt', data, (err) => {
            if (err) throw err;
            console.log("Data copied to destination.txt");

            // Verify by reading destination file
            fs.readFile('destination.txt', 'utf-8', (err, data1) => {
                if (err) throw err;
                console.log("Destination file content:", data1);
            });
        });
    });
});
```

Using copyFile method

```
const fs = require('fs');

// Step 1: Create source file
fs.writeFile('Source.txt', 'This is source file data.', (err) => {
    if (err) throw err;
    console.log("Source file created.");

    // Step 2: Copy file
    fs.copyFile('Source.txt', 'destination.txt', (err) => {
        if (err) throw err;
        console.log("File copied successfully.");

        // Step 3: Read destination file
        fs.readFile('destination.txt', 'utf-8', (err, data) => {
            if (err) throw err;
            console.log("Destination File Content:");
            console.log(data);
        });
    });
});
```

Write a node js script to write the text "You are creating a file" to help.txt file. After that append the text "you are appending data" to same help.txt file. After that read the file and print file contents on console. After finishing read operation , print the line "Thanks for using my program" on console. write ,append,read sequence must be maintain. all read ,write and append operations are asynchronous.

```
const fs = require('fs');
// Step 1: Write text to help.txt
fs.writeFile('help.txt', 'You are creating a file\n', (err) => {
   if (err) throw err;
   console.log("File created and data written.");
   // Step 2: Append text to help.txt
   fs.appendFile('help.txt', 'you are appending data\n', (err) => {
      if (err) throw err;
      console.log("Data appended.");
      // Step 3: Read the file
      fs.readFile('help.txt', 'utf-8', (err, data) => {
         if (err) throw err;
         console.log("\nFile Contents:");
         console.log(data);
         // Step 4: Final message
         console.log("Thanks for using my program");
      });
   });
});
```

OR Use setTimeout to read data

```
fs = require("fs")
fs.writeFile('help.txt', 'You are creating a file\n', function (err) {
   if (err) console.log("Error Generated"+err);
   fs.appendFile('help.txt', 'you are appending data\n', function (err) {
      if (err)
         console.log("Error Generated"+err);
   });
});
var rfile = function(){
   setTimeout(function() {
      fs.readFile('help.txt',"utf-8", (readErr, data) => {
         if (readErr) console.log("Error Generated: "+readErr)
         console.log(data);
         });
   },2000)
}
rfile();
```

Write file using one JSON Object and read file which gives you Same JSON object in console.

```
var ps=require("fs");
var data={"Name":"PKP"}
ps.writeFileSync("abc183.txt",JSON.stringify(data));
console.log("Entered data=")
console.log(data) //check same data will be stored in abc183.txt
var data2=ps.readFileSync("abc183.txt","utf-8");
console.log("Read data=")
var obj=JSON.parse(data2)
console.log(obj);
```

Output

```
Entered data=
{ Name: 'PKP' }
Read data=
{ Name: 'PKP' }
```

Write file using having one JSON array of two Object and read file which gives you Same JSON object in console.

```
var ps=require("fs");
var data={"Name":[{"Firstname":"Khushbu"},{"Lastname":"Patel"}]}

ps.writeFileSync("h1.txt",JSON.stringify(data));

console.log(data) // Data in file
console.log(data.Name[0].Firstname)
var data2=ps.readFileSync("h1.txt","utf-8");
var obj=JSON.parse(data2) //stored data in string formate so require to convert in object form/
console.log(obj.Name[0].Firstname +" "+ obj.Name[1].Lastname);
```

Output:

```
{ Name: [ { Firstname: 'Khushbu' }, { Lastname: 'Patel' } ] }

Khushbu

Khushbu Patel
```

Defining an array of object with properties name and age. Write this object in a file named student.txt then read the file and display the object on console.

```
const student =     [ { name: "ABC",  age: 30}, { name: "XYZ", age: 32} ]

var ps=require("fs");

ps.writeFileSync("student.txt",JSON.stringify(student));
data=ps.readFileSync("student.txt","utf-8");

b=JSON.parse(data);
console.log(b);


Output:
[ { name: 'ABC', age: 30 }, { name: 'XYZ', age: 32 } ]
```

Defining an array of object with properties name and age. Write this object in a file named student.txt then read the file and display the object on console.

```
const student =
   [
      {
         name: "ABC",
         age: 30
      },
      {
         name: "XYZ",
         age: 32
      }
   ]
var ps=require("fs");

ps.writeFileSync("student.txt",JSON.stringify(student));
data=ps.readFileSync("student.txt","utf-8");

b=JSON.parse(data);
console.log(b);


Output:
[ { name: 'ABC', age: 30 }, { name: 'XYZ', age: 32 } ]
```

Create JSON object which contains array of objects. Calculate perimeter of square and perimeter of circle by using side value and diameter value respectively. Write object as well as perimeter values of square and circle in shape.txt file.

```
const shape =
   [
      {
         name: "circle",
         diameter: 8
      },
      {
         name: "square",
         side: 10
      }
   ]
```

```
const shape =
   [
      {
         name: "circle",
         diameter: 8
      },
      {
         name: "square",
         side: 10
      }
   ]
var ps=require("fs");
ps.writeFileSync("shape.txt",JSON.stringify(shape));
data=ps.readFileSync("shape.txt","utf-8");
b=JSON.parse(data);

   var perimeter = (b[0].diameter/2) * 3.14 * 2 ; //2*3.14*r   (here r=diameter/2)
   console.log(perimeter);
   var peri = (b[1].side) *4  ; //4*side
   console.log(peri);

ps.appendFileSync("shape.txt","\nPerimeter of circle = "+ perimeter+ "\nPerimeter of square =
"+ peri);
```

Output:

```
[{"name":"circle","diameter":8},{"name":"square","side":10}]
Perimeter of circle = 25.12
Perimeter of square = 40
```

Write node js script and json to perform below tasks.

1. **Write below object in txt file named s2.txt**
   {d:{a:10,b:20,c:[30,10]}}
2. **Read data from the same file and perform the below tasks.**
   a. **addition of a and b.**
   b. **subtraction of 2nd element of c and b. (Must be positive value)**
   c. **multiplication of elements of c.**
3. **Add the Output of addition, subtraction and multiplication below the object in s2.txt file.**

```
var ps=require("fs");
const data ={d:{a:10,b:20,c:[30,10]}}

ps.writeFileSync("s2.txt",JSON.stringify(data)+"\n");
data1=ps.readFileSync("S2.txt","utf-8");
console.log(data1)
data1=JSON.parse(data1);
add=data1.d.a+data1.d.b;
sub=Math.abs(data1.d.c[1]-data1.d.b);
mul=data1.d.c[1]*data1.d.c[0];
console.log(add,sub,mul)
ps.appendFileSync("s2.txt",add+"\n"+sub+"\n"+mul)
```

❖ Path Module

The Path module provides a way of working with directories and file paths.

The syntax for including the path module in your application:
var pm=require("path");

Mehod	Description
.basename()	Returns the last part of a path
.dirname()	Returns the directories of a path
.extname()	Returns the file extension of a path
.parse()	Returns an object containing path information

Note: File path will be pass as a parameter inside method

Example:

```
var pm=require("path");
path1=pm.dirname("D:/FSD-2/node/addon.txt");
  console.log("Path: " + path1);
path2=pm.extname("D:/FSD-2/node/addon.txt");
  console.log("Extension: "+path2);
path2=pm.basename("D:/FSD-2/node/addon.txt");
  console.log("Basename: "+ path2);
path2=pm.parse("D:/FSD-2/node/addon.txt");
  console.log(path2);
  console.log(path2.root);
  console.log(path2.dir);
  console.log(path2.base);
  console.log(path2.ext);
  console.log(path2.name);
```

Output:

```
Path: D:/FSD-2/node
Extension: .txt
Basename: addon.txt
{
  root: 'D:/',
  dir: 'D:/FSD-2/node',
  base: 'addon.txt',
  ext: '.txt',
  name: 'addon'
}
D:/
D:/FSD-2/node
addon.txt
.txt
addon
// PS D:\Khushbu_Patel>
```

Write node.js script to check whether the file extension is .txt or not.

```
var pm=require("path");
path=pm.dirname("D:/LJ/abc.txt");
console.log(path);
path=pm.basename("D:/LJ/abc.txt");
console.log(path);
ext = pm.extname("D:/LJ/abc.txt")
console.log(ext);
path=pm.parse("D:/LJ/abc.html"); // It is not text file as changed to html
console.log(path);

if(path.ext == ".txt"){
   console.log("Text Document");
}else{
   console.log("Not a text Document");
}
```

Output:

```
D:/LJ
abc.txt
.txt
{
  root: 'D:/',
  dir: 'D:/LJ',
  base: 'abc.html',
  ext: '.html',
  name: 'abc'
}
Not a text Document
```

Write a Node.js script that performs the following operations using the Path module and the asynchronous File System (fs) module:

1. **From a given existing file path, extract the directory name using the Path module.**
2. **Create the extracted directory inside an existing folder using the File System module.**
3. **Extract the file name from the given path using the Path module.**
4. **Create that file inside the newly created directory and write some data into it.**
5. **Copy the content of this file to another file.**
6. **Delete the original file after copying the content.**

All file operations must be performed using asynchronous methods of the fs module.

```
const fs = require("fs");
const path = require("path");

// given existing path
```

```
let oldPath = "LJ/sample.txt";

// get directory from path
let dir = path.dirname(oldPath);

// get file name
let fileName = path.basename(oldPath);

// full file path
let newFilePath = dir + "/" + fileName;

/* Step 1: Create directory */
fs.mkdir(dir, (err) => {
    if (err) throw err;
    console.log("Directory created:", dir);

    /* Step 2: Create original file */
    fs.writeFile(newFilePath, "Sample Data for File", (err) => {
        if (err) throw err;
        console.log("Original file created:", newFilePath);

        /* Step 3: Copy file */
        fs.copyFile(newFilePath, dir + "/temp.txt", (err) => {
            if (err) throw err;
            console.log("File copied to temp.txt");

            /* Step 4: Delete original file */
            fs.unlink(newFilePath, (err) => {
                if (err) throw err;
                console.log("Original file deleted");

                console.log("All operations completed successfully");
            });
        });
    });
});
```

❖ Events

- Every action on a computer is like an event. For, e.g., when a connection is made, or you open any file.
- Node.js is also called an asynchronous event-driven JavaScript runtime.
- Its event-driven architecture allows it to perform asynchronous tasks.
- When a function (Callback) listens to a particular event to occur, all the callbacks subscribing to that event are fired one by one in the sequence in which they were registered.
- In Node.js, an event emitter is like a messenger that helps different parts of a program communicate with each other. Imagine it as a broadcaster at a big event, letting everyone know what's happening.
- If you worked with JavaScript in the browser, you know how much of the interaction of the user is handled through events: mouse clicks, keyboard button presses, reacting to mouse movements, and so on.
- On the backend side, Node.js offers us the option to build a similar system using the events module.
- It has a built-in module called the **'events'** module, where you can create, fire, and listen for your events.

The EventEmmitter class: All objects that emit events are instances of the EventEmitter class. The event can be emitted or listen to an event with the help of EventEmitter.

An EventEmitter object has two functions:

- Emit a named event.
- Attach or detach listeners to the named event.

Syntax:

```
const EventEmitter=require('events');
var eventEmitter=new EventEmitter();
```

Listening events: Before emits any event, it must register functions(callbacks) to listen to the events.

```
eventEmitter.addListener(event, listener)

eventEmitter.on(event, listener)

eventEmitter.once(event, listener)
```

1) eventEmitter.addListener(event, listener): It adds the listener at the end of the listener's array for the specified event. Multiple calls to the same event and listener will add the listener multiple times and correspondingly fire multiple times. Both functions return emitter, so calls can be chained.

2) eventEmmitter.on("event-name", callback): It is similar to eventEmmitter.addListner (event, listener) . This functions an event with its handler(listener) and calls the callback function once event is emitted.

3) eventEmitter.once(event, listener): It fires at most once for a particular event and will be removed from listeners array after it has listened once. Returns emitter, so calls can be chained.

Emitting events:

Every event is named event in nodejs. We can trigger an event by emit(event, [arg1], [arg2], […]) function. We can pass an arbitrary set of arguments to the listener functions.

Syntax:
eventEmitter.emit(event, [arg1], [arg2], [...])

Removing Listener:

The **eventEmitter.removeListener()** takes two argument event and listener, and removes that listener from the listeners array that is subscribed to that event. While **eventEmitter.removeAllListeners()** removes all the listener from the array which are subscribed to the mentioned event.

Syntax:
eventEmitter.removeListener(event, listener)

eventEmitter.removeAllListeners([event])

Note:

- Removing the listener from the array will change the sequence of the listener's array, hence it must be carefully used.
- The **eventEmitter.removeListener()** will remove at most one instance of the listener which is in front of the queue.

ListenerCount:

The **eventEmitter.listenerCount()** returns the number of listeners listening to the specified event.

Syntax:
eventEmitter.listenerCount(event)

Steps for event handling script

```
// 1) Import "events" module
var EventEmitter=require("events");
// 2) Create EventEmitter object
var ee=new EventEmitter();
// 3) Bind connection event with the handler/function
ee.on("sayName",()=>{
   console.log("your name is xyz")
});
// 4) Emit / Fire connection event
ee.emit("sayName");
```

- In this code, you first require the **events** module by assigning it to the variable **e**.
- Then, you create a new instance of the **EventEmitter** class and assign it to the variable **ee**.
- Next, you define an event listener for the event named **"sayName"**.
- When this event is emitted, the listener will be triggered, and it will log the message **"your name is xyz"** to the console.
- Finally, you emit the event **"sayName"** by calling **ee.emit("sayName")**. This will trigger the event listener, and you will see the corresponding message printed in the console: **"your name is xyz"**.

Position matters

```
var e=require("events");
var ee=new e();
ee.emit("sayName");
ee.on("sayName",()=>{
   console.log("your name is xyz")
});
```

*In this case, when you emit the event before defining the listener, there are no registered listeners yet, so nothing will happen.

Example: Registering for the event with call-back parameter.

```
var e=require("events");
var ee=new e();
ee.on("sayName",(statusCode,msg)=>{
   console.log(`status code id ${statusCode} and page is ${msg}`);
});
ee.emit("sayName",200,"ok");
```

Output:
status code id 200 and page is ok

Task: Write a Node.js program using the events module to simulate a sequence of events:

1. When a "connection" event occurs, print "Connection successfully" and trigger a "data-received" event.

2. When the "data-received" event occurs, print "Data received successfully".

3. Finally, print "Thanks" at the end of execution.

```
var e=require("events");
var ee=new e();
```

3) Generate Event Handler as follows:

```
var connectHandler = function connect(){ console.log("connection successful");
                                        ee.emit("data-received")      }//fire data-received event
```

4) Bind connection event with the handler

```
ee.on("connection",connectHandler);
```

5) Bind data-received event with an anonymous function

```
ee.on("data-received",function(){  console.log("data received successfully")})
```

6) Emit / Fire connection event

```
ee.emit('connection');
console.log("Thanks")
```

Output:

```
connection successfully

data received successfully

Thanks
```

Example: create an event emitter instance and register a couple of call-back.

```
var e=require("events");
var ee=new e();
ee.on("sayName",()=>{    console.log("your firstname is Khushbu")});
ee.on("sayName",()=>{    console.log("your middlename is P.")});
ee.on("sayName",()=>{    console.log("your lastname is Patel")});
ee.emit("sayName");
```

Output

```
your firstname is Khushbu

your middlename is P.

your lastname is Patel
```

Add / Remove Listener

Write a NodeJs script to create two listeners for a common event. Call their respective callbacks. Print no. of events associated with an emitter. Remove one of the listener and print no of remaining listeners.

```
var event = require('events');
var ee = new event.EventEmitter();

var listener1 = function listener1() {
  console.log("listener1 executed")
}

var listener2 = function listener2() {
  console.log("listener2 executed")
}
ee.addListener("conn",listener1)
ee.on("conn",listener2)

var count=ee.listenerCount("conn")          //counts listeners for conn event
console.log(count+" for conn event")
ee.emit("conn")

ee.removeListener("conn",listener1)        //remove listener1 form conn event
var count=ee.listenerCount("conn")
console.log(count+" for conn event")
ee.emit("conn")
```

// Above program ends here. Below is additional task of remove all listeners. and count the listener.

```
ee.removeAllListeners('conn', listener1);
count = ee.listenerCount("conn");
console.log("Again Count afetr removing all listeners: " + count );
//eventEmitter.emit("conn");
```

Output:

2 for conn event

listener1 executed

listener2 executed

1 for conn event

listener2 executed

//After removing all listeners
Again Count after removing all listeners: 0

Write a node js script to write the text "This is data" to new.txt file. After that append the text "that is data" to same ne .txt file. After that read the file and print file concept on console. After finishing read operation, print the line "Thanks for using my program" on console. All read/write operations are asynchronous. (Using Event)

```
const fs = require('fs');
const e = require('events');
const ee = new e();
ee.on('write', () => {
  fs.writeFile("new.txt", 'this is data', () => {
    console.log('Successfully wrote to file.');
    ee.emit('append'); // trigger event after write finishes
  });
});
ee.on('append', () => {
  fs.appendFile("new.txt", '\nthat is data', () => {
    console.log('Successfully appended to file.');
    ee.emit('read');
  });
});
ee.on('read', () => {
  fs.readFile("new.txt", 'utf8', (err, data) => {
if (err) throw err;
console.log(data);
    ee.emit('finish');
  });
});
ee.on('finish', () => {
  console.log('Thanks for using my program.');
});
ee.emit('write');  // Process Start
```

Write node js script to handle events as asked below.
1) Check the radius is negative or not. If negative then display message "Radius" must be positive" else calculate the perimeter of circle.
2) Check side is negative or not. If negative then display message "Side must be positive" else calculate the perimeter of square.

```
var e = require("events");
var ee = new e();

ee.on("negradius", () => {
   console.log("Radius must be positive");
});
ee.on("negside", sidehandler = () => {
   console.log("Side must be positive");
});
```

```
ee.on("findval", (r,s) => {

   if (r < 0) {
      ee.emit("negradius");
   }
else{
      var rperi = 2 * 3.14 * r;
      console.log(rperi);
   }
   if (s < 0) {
      ee.emit("negside");
   }
   else{
      var speri = 4*s;
      console.log(speri);
   }});
ee.emit("findval",6,9);
```

Write a node.js script using Event handling to perform following tasks in sequence:
1) Create a folder named abc.
2) Create a file in it named xyz.txt and enter data into it.
3) Append data to that file xyz.txt and print message "Data Appended Successfully".
4) Read the content of the file xyz.txt and print the content on console.
5) Copy content from xyz.txt to pqr.txt file.
6) Delete the file xyz.txt. And Lastly print the message "All operations performed successfully" on console.
Do all the operations of File using synchronous file system module.

```
var EventEmitter = require("events");
var fs = require("fs");
var ee = new EventEmitter();

/* Event 1: Create Folder */
ee.on("createFolder", function () {
   if (!fs.existsSync("abc")) {
      fs.mkdirSync("abc");
      console.log("Folder 'abc' created.");
   } else {
      console.log("Folder 'abc' already exists.");
   }
   ee.emit("createFile");
});

/* Event 2: Create File */
ee.on("createFile", function () {
   fs.writeFileSync("abc/xyz.txt", "This is the first line.\n");
```

```
    console.log("File xyz.txt created.");
    ee.emit("appendData");
});

/* Event 3: Append Data */
ee.on("appendData", function () {
    fs.appendFileSync("abc/xyz.txt", "This is appended data.\n");
    console.log("Data Appended Successfully");
    ee.emit("readFile");
});

/* Event 4: Read File */
ee.on("readFile", function () {
    var data = fs.readFileSync("abc/xyz.txt", "utf-8");
    console.log("File Content:");
    console.log(data);
    ee.emit("copyFile");
});

/* Event 5: Copy File */
ee.on("copyFile", function () {
    fs.copyFileSync("abc/xyz.txt", "abc/pqr.txt");
    console.log("File copied to pqr.txt");
    ee.emit("deleteFile");
});

/* Event 6: Delete File */
ee.on("deleteFile", function () {
    fs.unlinkSync("abc/xyz.txt");
    console.log("xyz.txt deleted.");
    console.log("All operations performed successfully");
});

/* Start the process */
ee.emit("createFolder");
```

Environment Variables

What are Environment Variables?

Environment variables are **values stored outside the program** that are used to configure an application while it runs.

They are accessed in Node.js using:

```
process.env.VARIABLE_NAME
```

Why Do We Use Environment Variables?

1. **Security**
   To store sensitive information like **passwords, API keys, tokens** instead of writing them in code.
2. **Different Environments**
   The same program can run in **development, testing, and production** with different settings.
3. **Avoid Hardcoding**
   Values like **port numbers or database URLs** should not be fixed inside the code.

When Do We Use Them?

Use environment variables when storing:

- Database credentials
- API keys
- Server port numbers
- Secret tokens
- Environment mode (development / production)

Step 1: Install dotenv

First install the package using npm:

```
npm install dotenv
```

This package allows Node.js to **read variables from the .env file**.

Step 2: Create a .env File

Inside your project folder, create a file named:
.env
Example:

```
PORT=3000
DB_PASSWORD=secret123
```

Step 3: Use dotenv in Your Code

```
require("dotenv").config();
console.log(process.env.PORT);
```

require("dotenv")
Imports the **dotenv module**.

.config()
Loads the variables from the .env file into **Node.js environment variables**.

process.env.PORT
Accesses the value of **PORT** stored in the .env file.